

Laboratoire 3

Optimisations de compilation

Départements : TIC
Unité d'enseignement HPC

Auteur : Urs Behrmann
Professeur : Alberto Dassatti
Assistant : Bruno Da Rocha Carvalho
Classe : HPC
Salle de labo : A09
Date : 25.03.2026

Contents

1	Sortir un appel de fonction invariant hors d'une boucle	3
2	Réduction de force : remplacer le modulo par un ET bit à bit	3
3	Éliminer un branchement avec CMOV	4

1 Sortir un appel de fonction invariant hors d'une boucle

Si une fonction est appelée dans une boucle mais que son résultat ne change pas d'une itération à l'autre (parce qu'elle ne dépend que de variables stables), la solution est simple : on l'appelle une seule fois avant la boucle et on garde le résultat dans une variable. On peut aussi marquer la fonction `inline` pour éviter le coût du *call/ret*.

Sans optimisation : Compiler Explorer — on voit `call compute` à chaque itération de la boucle.

Optimisé à la main : Compiler Explorer — `compute` n'est appelé qu'une fois avant la boucle, la boucle ne fait plus qu'écrire la valeur déjà calculée.

Avec -O2 : Compiler Explorer — le compilateur fait tout ça tout seul, et en plus il déroule la boucle (*loop unrolling*).

2 Réduction de force : remplacer le modulo par un ET bit à bit

L'opération modulo (%) est lente parce qu'elle revient à faire une division entière. Quand le diviseur est une puissance de 2, on peut la remplacer par un simple &. Par exemple, `x % 8` donne le même résultat que `x & 7` pour les entiers positifs, parce que masquer les 3 bits de poids faible suffit.

Attention : pour les entiers signés négatifs, les deux opérations ne donnent pas le même résultat. C'est pourquoi -O2 ne remplace pas bêtement le modulo par un AND ; il génère quand même une petite séquence de correction pour gérer les négatifs.

Sans optimisation : Compiler Explorer — le compilateur génère plusieurs instructions pour gérer le cas des négatifs.

Optimisé à la main : Compiler Explorer — en écrivant `x & 7` directement, on obtient une seule instruction AND.

Avec -O2 : Compiler Explorer — le compilateur garde la séquence de correction (il ne peut pas savoir que `x` est positif), mais il supprime le prologue/épilogue inutile.

3 Éliminer un branchement avec CMOV

Quand le processeur rencontre un `if`, il doit deviner quel chemin prendre (*branch prediction*). Si la prédiction est mauvaise, il perd plusieurs cycles. L'instruction **CMOV** (*Conditional MOVE*) fait un déplacement conditionnel sans sauter nulle part, ce qui évite ce problème.

En C, écrire un opérateur ternaire (`a > b ? a : b`) au lieu d'un `if/return` suffit souvent à pousser le compilateur à utiliser **CMOV**, même sans flag d'optimisation.

Sans optimisation : Compiler Explorer — le `if` produit un `jle` suivi d'un `jmp`.

Optimisé à la main : Compiler Explorer — l'opérateur ternaire génère un `cmovge`, plus de saut conditionnel.

Avec -O2 : Compiler Explorer — le compilateur supprime aussi la stack frame, résultat : trois instructions seulement.